Java Coding Interview Revision Book

60 worked problems for a CodeSignal-style screen

Final-year engineering level

Every problem has a complete Java solution, its complexity,
and the mistake that most often costs the points.

Companion to `ASSESSMENT.md` — work the book first, sit the assessment second.

How to use this book

The screen you are preparing for is reported to be four problems in seventy minutes on CodeSignal, with tasks in the range of *array manipulation* and *data processing: calculating averages, filtering data*. That is the bar. It is not a competitive-programming bar, and almost nothing in rounds like this needs dynamic programming or graph theory.

What it does need is that you write correct Java *quickly*, without stopping to remember how a `HashMap` iterates or what `split(",")` does to trailing empty fields. Speed on easy and medium problems is worth far more than the ability to crack one hard one.

A working method.

1. Read the whole statement before typing. Note every edge case it names: `null`, empty, ties, bounds. They are in the statement because they are in the tests.
2. Say the approach out loud in one sentence. If you cannot, you do not have one yet.
3. Write the happy path. Then go back and add the edge cases you noted.
4. Re-read the statement against your code before moving on. The most expensive bug in a timed test is the one you never look for.

How to drill. Give yourself the time budget printed on each difficulty: Easy 8 minutes, Medium 15, Hard 25. Write the solution without looking. Then read the **Trap** note — it is the thing that actually breaks submissions, and it is worth more than the solution itself.

Chapters 1 to 8 are the exam. Chapter 9 is deliberately above the bar. Work it only when the rest is comfortable; it is there so that a hard problem on the day does not surprise you.

Contents

How to use this book **1**

1 The Java toolkit **3**
Choosing the container . . . 3
Idioms worth memorising . . . 3
Traps that cost real points . . . 4
Complexity you should recognise instantly . . . 4

2 Strings **5**
P1. Reverse the words in a sentence . . . 5
P2. Valid palindrome, ignoring punctuation . . . 5
P3. First non-repeating character . . . 5
P4. Run-length encoding . . . 6
P5. Anagram check . . . 6
P6. Longest common prefix . . . 7
P7. Compare two version strings . . . 7
P8. Longest substring without repeating characters . . . 8

3 Arrays and two pointers **9**
P9. Move zeros to the end . . . 9
P10. Remove duplicates from a sorted array . . . 9
P11. Two-sum in a sorted array . . . 9
P12. Maximum subarray sum . . . 10
P13. Rotate an array by k . . . 10
P14. Merge two sorted arrays . . . 11
P15. Product of the array except self . . . 11
P16. Container with most water . . . 11

4 Hashing and counting **13**
P17. Count pairs summing to a target . . . 13
P18. Longest subarray with equal counts . . . 13
P19. Group anagrams . . . 14
P20. Top k frequent elements . . . 14
P21. Subarray sum equals k . . . 14
P22. First duplicate . . . 15
P23. Intersection of two arrays . . . 15
P24. Isomorphic strings . . . 16

5 Sorting, intervals and comparators **17**
P25. Sort by frequency, then by value . . . 17
P26. Merge overlapping intervals . . . 17
P27. Can one person attend all meetings? . . . 18
P28. Minimum number of meeting rooms . . . 18
P29. Insert an interval . . . 19
P30. Sort employees by salary then name . . . 19
P31. Kth largest element . . . 20

6 Matrices **21**
P32. Rotate a square matrix 90 degrees clockwise . . . 21
P33. Spiral traversal . . . 21
P34. Count islands . . . 22
P35. Set matrix zeroes . . . 22
P36. Diagonal sums . . . 23
P37. Search a row-and-column sorted matrix . . . 23

7 Recursion, BFS and DFS **24**
P38. Fibonacci with memoisation . . . 24
P39. Maximum depth of a binary tree . . . 24
P40. Level-order traversal . . . 25
P41. Flood fill . . . 25
P42. All permutations of a string . . . 26
P43. All subsets . . . 26
P44. Shortest path in a grid with obstacles . . . 27

8 Stacks and queues **28**
P45. Valid parentheses . . . 28
P46. Evaluate reverse Polish notation . . . 28
P47. Next greater element . . . 29
P48. Min stack . . . 29
P49. Simplify a Unix path . . . 30

9 Data processing **31**
P50. Average salary by department . . . 31
P51. Filter and total transactions . . . 32
P52. Validate records and report the errors . . . 32
P53. Join two datasets on a key . . . 33
P54. Bucket events by hour and find the peak . . . 33
P55. Running balance per account . . . 34

10 Stretch **36**
P56. Minimum travel time with k free roads . . . 36
P57. Expiring files . . . 37
P58. LRU cache . . . 37
P59. Coin change . . . 38
P60. Longest increasing subsequence . . . 39

1. The Java toolkit

The twenty things you must not have to think about on the day. Read this once; come back to it whenever a drill makes you hesitate.

Choosing the container

You need	Use	Why / cost
Count things by key	`HashMap<K,Integer>`	`merge(k,1,Integer::sum)`, O(1)
Group things by key	`HashMap<K,List<V>>`	`computeIfAbsent(k, x -> new ArrayList<>())`
Seen-before / dedupe	`HashSet<T>`	`add` returns `false` if present
Keys in sorted order	`TreeMap<K,V>`	O(log n), gives `firstKey`, `floorKey`
Insertion order kept	`LinkedHashMap`	iteration order = insertion order
Growable array	`ArrayList<T>`	O(1) amortised add, O(1) get
Stack or queue	`ArrayDeque<T>`	`push/pop`, `offer/poll`; never `Stack`
Top-k / smallest-first	`PriorityQueue<T>`	O(log n) per op, min-heap by default

`ArrayDeque` is the right answer for both stacks and queues. `java.util.Stack` is a synchronised relic and `LinkedList` costs you a pointer chase per element.

Idioms worth memorising

```
// count occurrences
Map<Character,Integer> freq = new HashMap<>();
for (char c : s.toCharArray()) freq.merge(c, 1, Integer::sum);

// group into lists
Map<String,List<String>> byKey = new HashMap<>();
byKey.computeIfAbsent(key, k -> new ArrayList<>()).add(value);

// accumulate a total and a count together
map.computeIfAbsent(dept, k -> new long[2])[0] += salary;   // [0]=total
map.get(dept)[1]++;                                          // [1]=count

// sort with a tie-break: primary descending, secondary ascending
list.sort((a, b) -> a.score != b.score
        ? Integer.compare(b.score, a.score)
        : a.name.compareTo(b.name));

// iterate a map
for (Map.Entry<String,Integer> e : map.entrySet())
    use(e.getKey(), e.getValue());

// array <-> list
int[] arr = list.stream().mapToInt(Integer::intValue).toArray();
String[] out = list.toArray(new String[0]);

// 2D array, all cells set
char[][] g = new char[rows][cols];
for (char[] row : g) Arrays.fill(row, '.');

// the four neighbours (eight: nest -1..1 and skip 0,0)
int[][] DIRS = {{-1,0},{1,0},{0,-1},{0,1}};
```

Traps that cost real points

- **`split(",")` drops trailing empties.** `"a,b,"` splits into 2 parts, not 3. Use `split(",", -1)` whenever the field count is part of your validation.
- **`%` keeps the sign of the dividend.** `-3 % 2 == -1`, so `n % 2 == 1` is false for negative odd numbers. Write `n % 2 != 0`.
- **Integer division truncates toward zero.** `7/2 == 3` and `-7/2 == -3`. That is not floor for negatives.
- **`int` overflow is silent.** Sums of up to 2×10^9 values overflow. Accumulate in `long`.
- **`==` on boxed types compares references.** Use `.equals`, or unbox to `int` first. `Integer` caches only -128 to 127, which is why small cases seem to work.
- **`String +=` in a loop is $O(n^2)$.** Use `StringBuilder`.
- **`ConcurrentModificationException`** when you add to or remove from a collection while a for-each is running. Collect the keys first, then mutate; or use `Iterator.remove()`.
- **A recursive flood fill overflows the stack** at a few tens of thousands of cells. Use an explicit `ArrayDeque`.
- **`Arrays.sort` on primitives is a dual-pivot quicksort** with an $O(n^2)$ worst case on crafted input; on objects it is a stable merge sort. Boxing to `Integer[]` is the paranoid fix, and almost never needed.
- **Returning `null` where an empty array was asked for.** Read the return contract twice.

Complexity you should recognise instantly

Shape	Cost	Safe up to
Single pass	$O(n)$	10^8
Sort	$O(n \log n)$	10^7
Nested loop over the input	$O(n^2)$	$\sim 10^4$
Pass per element, each a scan	$O(n^2)$	the same, and this is the one you write by accident
Flood fill / BFS on a grid	$O(\text{rows} \times \text{cols})$	10^7 cells

If the constraints say $n \leq 200\,000$, an $O(n^2)$ solution will time out. That number in a statement is the examiner telling you the intended complexity.

2. Strings

Eight problems. Strings are the most common opener in a screen because they are quick to state and full of edge cases.

P1. Reverse the words in a sentence

Easy

Given a sentence, return it with the words in reverse order and exactly one space between them. Leading, trailing and repeated spaces are removed. `" the sky  is blue "` becomes `"blue is sky the"`.

Idea. Split on runs of whitespace, walk the parts backwards, join with a single space.

Solution

```java
public static String reverseWords(String s) {
    if (s == null) return "";
    String[] parts = s.trim().split("\\s+");
    StringBuilder out = new StringBuilder();
    for (int i = parts.length - 1; i >= 0; i--) {
        if (parts[i].isEmpty()) continue;
        if (out.length() > 0) out.append(' ');
        out.append(parts[i]);
    }
    return out.toString();
}
```

Cost. $O(n)$ time, $O(n)$ space.

Trap. Splitting on `" "` instead of `"\\s+"` leaves empty strings in the array when spaces repeat. And `trim()` first, or a leading space gives you an empty first element anyway.

P2. Valid palindrome, ignoring punctuation

Easy

Return `true` if the string reads the same forwards and backwards when you consider only letters and digits, and ignore case. `"A man, a plan, a canal:  Panama"` is a palindrome.

Idea. Two pointers walking inwards, each skipping anything that is not alphanumeric.

Solution

```java
public static boolean isPalindrome(String s) {
    if (s == null) return true;
    int i = 0, j = s.length() - 1;
    while (i < j) {
        while (i < j && !Character.isLetterOrDigit(s.charAt(i))) i++;
        while (i < j && !Character.isLetterOrDigit(s.charAt(j))) j--;
        if (Character.toLowerCase(s.charAt(i)) !=
            Character.toLowerCase(s.charAt(j))) return false;
        i++; j--;
    }
    return true;
}
```

Cost. $O(n)$ time, $O(1)$ space.

Trap. The inner skip loops need their own `i < j` guard, or a string of pure punctuation runs the pointer off the end.

P3. First non-repeating character

Easy

Return the index of the first character that appears exactly once, or -1. In `"loveleetcode"` the answer is 2.

Idea. Count every character in one pass, then scan the string again in order and return the first with count 1. Two passes beat one clever pass.

Solution

```
public static int firstUnique(String s) {
    if (s == null || s.isEmpty()) return -1;
    int[] freq = new int[128];                  // ASCII
    for (int i = 0; i < s.length(); i++) freq[s.charAt(i)]++;
    for (int i = 0; i < s.length(); i++)
        if (freq[s.charAt(i)] == 1) return i;
    return -1;
}
```

Cost. $O(n)$ time, $O(1)$ space (the table is a fixed 128).

Trap. Scanning the *map* rather than the string for the second pass loses the ordering. `HashMap` has no order; if you must use one, use `LinkedHashMap`, or just re-scan the string as above.

P4. Run-length encoding

Easy

Replace each maximal run of identical characters with the character followed by the run length, writing a run of one as just the character. `"aaabccddd"` becomes `"a3bc2d3"`.

Idea. One pointer at the start of the run, a second scanning forward while the character is unchanged.

Solution

```
public static String compress(String s) {
    if (s == null || s.isEmpty()) return "";
    StringBuilder out = new StringBuilder();
    int i = 0;
    while (i < s.length()) {
        char ch = s.charAt(i);
        int j = i;
        while (j < s.length() && s.charAt(j) == ch) j++;
        out.append(ch);
        if (j - i > 1) out.append(j - i);
        i = j;
    }
    return out.toString();
}
```

Cost. $O(n)$ time, $O(n)$ space.

Trap. Run lengths can have more than one digit — `"z12"` is a valid output. Any solution that appends a single `char` for the count is wrong past nine.

Also appears as: Task 1 of the assessment.

P5. Anagram check

Easy

Return `true` if two strings contain the same letters with the same multiplicities.

Idea. Count up for the first string, count down for the second, and check nothing is left over. Length differing is an instant `false`.

Solution

```
public static boolean isAnagram(String a, String b) {
    if (a == null || b == null || a.length() != b.length()) return false;
    int[] freq = new int[26];
    for (int i = 0; i < a.length(); i++) {
        freq[a.charAt(i) - 'a']++;
        freq[b.charAt(i) - 'a']--;
    }
```

```
    for (int f : freq) if (f != 0) return false;
    return true;
}
```

Cost. $O(n)$ time, $O(1)$ space.

Trap. The `- 'a'` indexing assumes lowercase letters only. If the input can be any character, use a `HashMap` or a 128-entry table — an uppercase letter would write to a negative index and throw.

P6. Longest common prefix

Easy

Given an array of strings, return the longest prefix common to all of them, or `""`.

Idea. Compare column by column: take character i of the first string and check it against character i of every other. Stop at the first mismatch or the first string that ends.

Solution

```
public static String longestCommonPrefix(String[] words) {
    if (words == null || words.length == 0) return "";
    for (int i = 0; i < words[0].length(); i++) {
        char c = words[0].charAt(i);
        for (int w = 1; w < words.length; w++) {
            if (words[w] == null) return "";
            if (i >= words[w].length() || words[w].charAt(i) != c)
                return words[0].substring(0, i);
        }
    }
    return words[0];
}
```

Cost. O(total characters) time, $O(1)$ extra space.

Trap. Forgetting the `i >= words[w].length()` check throws `StringIndexOutOfBounds` as soon as one word is a strict prefix of another (`["ab", "a"]`).

P7. Compare two version strings

Medium

Versions are dot-separated numbers: `"1.2.10"`. Return -1, 0 or 1 for less than, equal, greater than. Missing components count as zero, so `"1.0"` equals `"1"`.

Idea. Split both, then walk to the longer length, treating a missing component as zero. Compare numerically, never as text.

Solution

```
public static int compareVersions(String a, String b) {
    String[] x = a.split("\\.");
    String[] y = b.split("\\.");
    int n = Math.max(x.length, y.length);
    for (int i = 0; i < n; i++) {
        long left  = i < x.length ? Long.parseLong(x[i]) : 0;
        long right = i < y.length ? Long.parseLong(y[i]) : 0;
        if (left != right) return left < right ? -1 : 1;
    }
    return 0;
}
```

Cost. $O(n)$ time.

Trap. Two of them. `split(".")` splits on *every character*, because `.` is a regex wildcard — you must escape it. And comparing the components as strings makes `"1.10"` less than `"1.9"`.

P8. Longest substring without repeating characters

Medium

Return the length of the longest substring in which no character repeats. In `"abcabcbb"` the answer is 3.

Idea. A sliding window. Remember the last index at which each character was seen; when you meet a repeat that is inside the window, jump the left edge past it.

Solution

```
public static int longestUnique(String s) {
    if (s == null || s.isEmpty()) return 0;
    Map<Character,Integer> lastSeen = new HashMap<>();
    int best = 0, left = 0;
    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        Integer prev = lastSeen.get(c);
        if (prev != null && prev >= left) left = prev + 1;
        lastSeen.put(c, right);
        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

Cost. $O(n)$ time, $O(\min(n, \Sigma))$ space.

Trap. The `prev >= left` test is the whole problem. Without it a character seen long ago, already outside the window, drags the left edge *backwards* and the window stops being valid.

3. Arrays and two pointers

Eight problems. The two-pointer idea — a read index and a write index, or two ends closing in — turns a great many quadratic solutions linear.

P9. Move zeros to the end

Easy

Move every zero to the end of the array, keeping the order of the non-zero elements. Do it in place. `[0,1,0,3,12]` becomes `[1,3,12,0,0]`.

Idea. A write pointer. Copy every non-zero forward to the write position, then fill the tail with zeros.

Solution

```
public static void moveZeros(int[] a) {
    if (a == null) return;
    int write = 0;
    for (int read = 0; read < a.length; read++)
        if (a[read] != 0) a[write++] = a[read];
    while (write < a.length) a[write++] = 0;
}
```

Cost. $O(n)$ time, $O(1)$ space.

Trap. Swapping instead of copying also works, but only if you swap with the write pointer and not with the end of the array — swapping with the end destroys the relative order.

P10. Remove duplicates from a sorted array

Easy

The array is sorted ascending. Compact it in place so each value appears once, and return the new length. The elements past that length do not matter.

Idea. Same write pointer. Because the array is sorted, a duplicate is always adjacent to its twin, so compare each element with the last one you kept.

Solution

```
public static int dedupeSorted(int[] a) {
    if (a == null || a.length == 0) return 0;
    int write = 1;
    for (int read = 1; read < a.length; read++)
        if (a[read] != a[write - 1]) a[write++] = a[read];
    return write;
}
```

Cost. $O(n)$ time, $O(1)$ space.

Trap. Comparing `a[read]` with `a[read-1]` looks equivalent and is not: once you start overwriting, `a[read-1]` may be a value you already moved. Compare against the last *kept* element.

P11. Two-sum in a sorted array

Easy

Given a sorted array and a target, return the indices of two values that add to the target, or `null`.

Idea. One pointer at each end. If the sum is too big the right pointer must come in; if too small the left must go out. No other move can help.

Solution

```
public static int[] twoSumSorted(int[] a, int target) {
    if (a == null) return null;
    int i = 0, j = a.length - 1;
```

```
    while (i < j) {
        long sum = (long) a[i] + a[j];
        if (sum == target) return new int[] {i, j};
        if (sum < target) i++; else j--;
    }
    return null;
}
```

Cost. $O(n)$ time, $O(1)$ space. On an *unsorted* array use a `HashMap` of value to index instead — also $O(n)$.

Trap. `a[i] + a[j]` can overflow `int` with large values. Widen to `long` before adding.

P12. Maximum subarray sum

Medium

Return the largest sum obtainable from a contiguous, non-empty subarray.
`[-2,1,-3,4,-1,2,1,-5,4]` gives 6.

Idea. Kadane. At each element, the best subarray ending here is either this element alone or this element extending the best ending just before it.

Solution

```
public static int maxSubarraySum(int[] a) {
    if (a == null || a.length == 0) return 0;
    long best = a[0], endingHere = a[0];
    for (int i = 1; i < a.length; i++) {
        endingHere = Math.max(a[i], endingHere + a[i]);
        best = Math.max(best, endingHere);
    }
    return (int) best;
}
```

Cost. $O(n)$ time, $O(1)$ space.

Trap. Initialising `best` to `0` instead of `a[0]` returns 0 for an all-negative array, when the answer is the least negative element. The problem says non-empty.

P13. Rotate an array by k

Medium

Shift every element right by k positions, wrapping around, in place. `[1,2,3,4,5]` with $k = 2$ becomes `[4,5,1,2,3]`.

Idea. Reverse the whole array, then reverse the first k and the rest separately. Three reversals, no extra array.

Solution

```
public static void rotate(int[] a, int k) {
    if (a == null || a.length == 0) return;
    k = ((k % a.length) + a.length) % a.length;   // handles negative k
    reverse(a, 0, a.length - 1);
    reverse(a, 0, k - 1);
    reverse(a, k, a.length - 1);
}

private static void reverse(int[] a, int i, int j) {
    while (i < j) {
        int t = a[i]; a[i] = a[j]; a[j] = t;
        i++; j--;
    }
}
```

Cost. $O(n)$ time, $O(1)$ space.

Trap. k can exceed the length, and can be negative. The double-modulo normalises both. Without it, `k = 7` on a length-5 array throws.

P14. Merge two sorted arrays

Easy

Both inputs are sorted ascending. Return one sorted array containing all the elements.

Idea. One index into each, repeatedly take the smaller head. Then drain whichever array still has elements.

Solution

```java
public static int[] merge(int[] a, int[] b) {
    if (a == null) a = new int[0];
    if (b == null) b = new int[0];
    int[] out = new int[a.length + b.length];
    int i = 0, j = 0, k = 0;
    while (i < a.length && j < b.length)
        out[k++] = (a[i] <= b[j]) ? a[i++] : b[j++];
    while (i < a.length) out[k++] = a[i++];
    while (j < b.length) out[k++] = b[j++];
    return out;
}
```

Cost. $O(n + m)$ time, $O(n + m)$ space.

Trap. Forgetting the two drain loops. The main loop stops as soon as *either* array is exhausted, and the remainder of the other one still has to be copied.

P15. Product of the array except self

Medium

Return an array where position i holds the product of every element *except* `a[i]`. Do it without division. `[1,2,3,4]` gives `[24,12,8,6]`.

Idea. Two sweeps. Left to right, store the product of everything before i. Right to left, multiply in the product of everything after i, carried in a single variable.

Solution

```java
public static int[] productExceptSelf(int[] a) {
    if (a == null || a.length == 0) return new int[0];
    int n = a.length;
    int[] out = new int[n];
    out[0] = 1;
    for (int i = 1; i < n; i++) out[i] = out[i - 1] * a[i - 1];
    int suffix = 1;
    for (int i = n - 1; i >= 0; i--) {
        out[i] *= suffix;
        suffix *= a[i];
    }
    return out;
}
```

Cost. $O(n)$ time, $O(1)$ extra space beyond the output.

Trap. The division shortcut — total product divided by `a[i]` — breaks on a single zero and is undefined on two. The problem forbids it for exactly that reason.

P16. Container with most water

Medium

Each element is the height of a vertical line. Pick two lines so that the rectangle between them holds the most water: area is the shorter height times the distance.

Idea. Start at the widest pair and close in. The area is limited by the shorter line, so moving the *taller* one inwards can never help — width shrinks and the limit does not rise. Move the shorter one.

Solution

```java
public static int maxArea(int[] height) {
    if (height == null || height.length < 2) return 0;
    int i = 0, j = height.length - 1, best = 0;
    while (i < j) {
        int h = Math.min(height[i], height[j]);
        best = Math.max(best, h * (j - i));
        if (height[i] < height[j]) i++; else j--;
    }
    return best;
}
```

Cost. $O(n)$ time, $O(1)$ space.

Trap. Moving whichever pointer you feel like, or both, breaks the argument. The proof that nothing is missed depends on always discarding the shorter line.

4. Hashing and counting

Eight problems. If a question asks about how many, have I seen, *or* group by, *the answer starts with a* `HashMap`. *The prefix-sum trick in P18 and P21 is the single highest value pattern in this book.*

P17. Count pairs summing to a target

Easy

Count the pairs of indices $i < j$ with `a[i] + a[j] == target`. `[1,5,7,1]` with target 6 gives 2.

Idea. Walk once. For each element, the number of earlier elements that complete the pair is already in the map. Count first, then insert — that ordering is what stops an element pairing with itself.

Solution

```java
public static long countPairs(int[] a, int target) {
    if (a == null) return 0;
    Map<Integer,Integer> seen = new HashMap<>();
    long pairs = 0;
    for (int x : a) {
        pairs += seen.getOrDefault(target - x, 0);
        seen.merge(x, 1, Integer::sum);
    }
    return pairs;
}
```

Cost. $O(n)$ time, $O(n)$ space.

Trap. Inserting before counting makes `[3]` with target 6 report one pair. And the count can exceed `int` on a large array of equal values: $n = 200\,000$ identical elements is about 2×10^{10} pairs.

P18. Longest subarray with equal counts

Medium

Return the length of the longest contiguous subarray holding as many even numbers as odd ones. `[1,2,3,4,5]` gives 4.

Idea. Turn it into a running balance: $+1$ for odd, -1 for even. A subarray is balanced exactly when the balance at its two ends is equal. So record, for each balance value, the *earliest* index where it appeared; the longest span to a repeat of that value is the answer.

Solution

```java
public static int longestBalanced(int[] nums) {
    if (nums == null || nums.length == 0) return 0;
    Map<Integer,Integer> firstIndex = new HashMap<>();
    firstIndex.put(0, -1);                 // empty prefix
    int balance = 0, best = 0;
    for (int i = 0; i < nums.length; i++) {
        balance += (nums[i] % 2 != 0) ? 1 : -1;
        Integer seen = firstIndex.get(balance);
        if (seen == null) firstIndex.put(balance, i);
        else best = Math.max(best, i - seen);
    }
    return best;
}
```

Cost. $O(n)$ time, $O(n)$ space.

Trap. Three in one problem. Overwriting the stored index on a repeat measures the *shortest* span. Omitting the `0 -> -1` seed loses every answer that starts at index 0. And `nums[i] % 2 == 1` is false for -3.

Also appears as: Task 4 of the assessment; and, with 0 and 1 in place of even and odd, the classic "contiguous array" question.

P19. Group anagrams

Medium

Group the words that are anagrams of one another.
`["eat","tea","tan","ate"]` gives `[[eat,tea,ate],[tan]]`.
Idea. Give every word a canonical key that all its anagrams share. Sorting the letters is the obvious one; a 26-slot count rendered as a string is faster.

Solution

```java
public static List<List<String>> groupAnagrams(String[] words) {
    Map<String,List<String>> groups = new HashMap<>();
    for (String w : words) {
        char[] letters = w.toCharArray();
        Arrays.sort(letters);
        String key = new String(letters);
        groups.computeIfAbsent(key, k -> new ArrayList<>()).add(w);
    }
    return new ArrayList<>(groups.values());
}
```

Cost. $O(n\,k \log k)$ for n words of length k. The counting key makes it $O(nk)$.

Trap. Using the word's `hashCode` or its character *sum* as the key. Both collide: `"ad"` and `"bc"` have the same letter sum.

P20. Top k frequent elements

Medium

Return the k most frequent values, most frequent first.
Idea. Count, then select. A heap of size k keeps the selection at $O(n \log k)$; sorting the distinct values is simpler and usually fast enough.

Solution

```java
public static int[] topKFrequent(int[] a, int k) {
    Map<Integer,Integer> freq = new HashMap<>();
    for (int x : a) freq.merge(x, 1, Integer::sum);

    PriorityQueue<Map.Entry<Integer,Integer>> heap =
        new PriorityQueue<>(Map.Entry.comparingByValue());   // min-heap
    for (Map.Entry<Integer,Integer> e : freq.entrySet()) {
        heap.offer(e);
        if (heap.size() > k) heap.poll();     // drop the least frequent
    }
    int[] out = new int[heap.size()];
    for (int i = out.length - 1; i >= 0; i--) out[i] = heap.poll().getKey();
    return out;
}
```

Cost. $O(n \log k)$ time, $O(n)$ space.

Trap. Java's `PriorityQueue` is a *min*-heap. To keep the top k you push everything and evict the smallest, which is why the output has to be filled backwards. Reaching for a max-heap and polling k times also works but costs $O(n \log n)$.

P21. Subarray sum equals k

Medium

Count the contiguous subarrays whose elements sum to exactly k. Values may be negative.
Idea. Prefix sums. The sum of the window $(j, i]$ is $\text{prefix}_i - \text{prefix}_j$, so a window sums to k whenever an earlier prefix equals $\text{prefix}_i - k$. Count how many times each prefix has occurred.

Solution

```java
public static int subarraysSummingTo(int[] a, int k) {
    Map<Long,Integer> counts = new HashMap<>();
    counts.put(0L, 1);                    // the empty prefix
    long prefix = 0;
    int total = 0;
    for (int x : a) {
        prefix += x;
        total += counts.getOrDefault(prefix - k, 0);
        counts.merge(prefix, 1, Integer::sum);
    }
    return total;
}
```

Cost. $O(n)$ time, $O(n)$ space.

Trap. A sliding window does *not* work here, because negative values mean growing the window can lower the sum. Sliding windows need monotonicity; prefix sums do not.

P22. First duplicate

Easy

Return the first value that appears twice, scanning left to right, or -1.

Idea. A `HashSet`. `add` returns `false` when the element was already present, so the check and the insert are one call.

Solution

```java
public static int firstDuplicate(int[] a) {
    Set<Integer> seen = new HashSet<>();
    for (int x : a) if (!seen.add(x)) return x;
    return -1;
}
```

Cost. $O(n)$ time, $O(n)$ space.

Trap. "First duplicate" means the first value whose *second* occurrence comes earliest, which is what this returns. If the question instead wants the duplicate whose first occurrence is earliest, that is a different answer and needs a second pass.

P23. Intersection of two arrays

Easy

Return the values present in both arrays, each once.

Idea. Put the smaller array in a set, then filter the larger one, moving each hit into a result set so duplicates collapse.

Solution

```java
public static int[] intersection(int[] a, int[] b) {
    Set<Integer> inA = new HashSet<>();
    for (int x : a) inA.add(x);
    Set<Integer> both = new LinkedHashSet<>();
    for (int x : b) if (inA.contains(x)) both.add(x);
    int[] out = new int[both.size()];
    int i = 0;
    for (int x : both) out[i++] = x;
    return out;
}
```

Cost. $O(n + m)$ time, $O(n)$ space.

Trap. Collecting into a `List` gives duplicates when `b` repeats a shared value. `LinkedHashSet` also fixes the output order, which plain `HashSet` leaves unspecified — graders often compare arrays exactly.

P24. Isomorphic strings

Medium

Two strings are isomorphic if the characters of one can be mapped to the characters of the other consistently and one-to-one. `"egg"` and `"add"` are; `"foo"` and `"bar"` are not.

Idea. Two maps, one in each direction. A single map allows two different characters to map onto the same target, which breaks one-to-one.

Solution

```java
public static boolean isIsomorphic(String s, String t) {
    if (s == null || t == null || s.length() != t.length()) return false;
    Map<Character,Character> forward = new HashMap<>();
    Map<Character,Character> backward = new HashMap<>();
    for (int i = 0; i < s.length(); i++) {
        char a = s.charAt(i), b = t.charAt(i);
        Character f = forward.putIfAbsent(a, b);
        Character g = backward.putIfAbsent(b, a);
        if ((f != null && f != b) || (g != null && g != a)) return false;
    }
    return true;
}
```

Cost. $O(n)$ time, $O(1)$ space for a bounded alphabet.

Trap. Comparing the boxed `Character` values with `!=` works here only because the cache covers ASCII. Make it a habit to unbox or use `.equals` — the same code with `Integer` keys above 127 would silently fail.

5. Sorting, intervals and comparators

Seven problems. Writing a correct multi-key comparator under time pressure is a skill in itself, and interval sweeps are the most common "medium" in a screen.

P25. Sort by frequency, then by value

Medium

Sort the array so that more frequent values come first, and equally frequent values come in ascending numeric order. `[1,1,2,2,2,3]` gives `[2,2,2,1,1,3]`.

Idea. Count first, then sort the *boxed* values with a two-key comparator. Primitive arrays cannot take a comparator, so boxing is required.

Solution

```java
public static int[] sortByFrequency(int[] a) {
    Map<Integer,Integer> freq = new HashMap<>();
    for (int x : a) freq.merge(x, 1, Integer::sum);

    Integer[] boxed = new Integer[a.length];
    for (int i = 0; i < a.length; i++) boxed[i] = a[i];

    Arrays.sort(boxed, (x, y) -> {
        int fx = freq.get(x), fy = freq.get(y);
        if (fx != fy) return Integer.compare(fy, fx);   // frequency desc
        return Integer.compare(x, y);                   // value asc
    });

    int[] out = new int[a.length];
    for (int i = 0; i < a.length; i++) out[i] = boxed[i];
    return out;
}
```

Cost. $O(n \log n)$ time, $O(n)$ space.

Trap. `Arrays.sort(int[], Comparator)` does not exist — there is no such overload, and the compile error surprises people under pressure. Box to `Integer[]` or sort a `List<Integer>`.

P26. Merge overlapping intervals

Medium

Given intervals as `[start, end]`, merge every pair that overlaps. `[[1,3],[2,6],[8,10]]` gives `[[1,6],[8,10]]`.

Idea. Sort by start. Then one pass: either the next interval starts inside the one you are building, in which case stretch its end, or it does not, in which case close the current one and open a new one.

Solution

```java
public static int[][] merge(int[][] intervals) {
    if (intervals == null || intervals.length == 0) return new int[0][];
    int[][] a = intervals.clone();
    Arrays.sort(a, (x, y) -> Integer.compare(x[0], y[0]));

    List<int[]> out = new ArrayList<>();
    int[] current = new int[] {a[0][0], a[0][1]};
    for (int i = 1; i < a.length; i++) {
        if (a[i][0] <= current[1]) {
            current[1] = Math.max(current[1], a[i][1]);
        } else {
            out.add(current);
            current = new int[] {a[i][0], a[i][1]};
        }
```

```
    }
    out.add(current);
    return out.toArray(new int[0][]);
}
```

Cost. $O(n \log n)$ time, $O(n)$ space.

Trap. `current[1] = a[i][1]` without the `Math.max` loses a fully contained interval: merging `[1,10]` with `[2,3]` must still end at 10. Also remember to add the last `current` after the loop.

P27. Can one person attend all meetings? Easy

Given meeting intervals, return `true` if none of them overlap.

Idea. Sort by start; any overlap is then between neighbours.

Solution

```
public static boolean canAttendAll(int[][] meetings) {
    if (meetings == null || meetings.length < 2) return true;
    int[][] a = meetings.clone();
    Arrays.sort(a, (x, y) -> Integer.compare(x[0], y[0]));
    for (int i = 1; i < a.length; i++)
        if (a[i][0] < a[i - 1][1]) return false;
    return true;
}
```

Cost. $O(n \log n)$ time.

Trap. Whether touching intervals count as overlapping. `[1,5]` and `[5,9]` do not overlap if the end is exclusive — hence `<` and not `<=`. The statement decides; read it.

P28. Minimum number of meeting rooms Medium

Return the largest number of meetings running at the same time.

Idea. A sweep. Put every start and every end on a timeline, sort both, then walk them: a start raises the count, an end lowers it. Track the peak. Ends must be processed before starts at the same instant, since a room freed at 5 can be reused at 5.

Solution

```
public static int minRooms(int[][] meetings) {
    if (meetings == null || meetings.length == 0) return 0;
    int n = meetings.length;
    int[] starts = new int[n], ends = new int[n];
    for (int i = 0; i < n; i++) {
        starts[i] = meetings[i][0];
        ends[i]   = meetings[i][1];
    }
    Arrays.sort(starts);
    Arrays.sort(ends);

    int rooms = 0, peak = 0, e = 0;
    for (int s = 0; s < n; s++) {
        while (e < n && ends[e] <= starts[s]) { rooms--; e++; }
        rooms++;
        peak = Math.max(peak, rooms);
    }
    return peak;
}
```

Cost. $O(n \log n)$ time, $O(n)$ space.

Trap. `ends[e] < starts[s]` instead of `<=` books a second room for back-to-back meetings. The two arrays are sorted independently on purpose — the pairing between a particular start and end does not matter.

P29. Insert an interval

Medium

The intervals are already sorted and non-overlapping. Insert a new one, merging where it overlaps.

Idea. Three phases, no sorting needed: copy everything that ends before the new one starts, absorb everything that overlaps into a widened interval, then copy the rest.

Solution

```java
public static int[][] insert(int[][] intervals, int[] newInterval) {
    List<int[]> out = new ArrayList<>();
    int i = 0, n = intervals.length;
    int start = newInterval[0], end = newInterval[1];

    while (i < n && intervals[i][1] < start) out.add(intervals[i++]);

    while (i < n && intervals[i][0] <= end) {
        start = Math.min(start, intervals[i][0]);
        end   = Math.max(end,   intervals[i][1]);
        i++;
    }
    out.add(new int[] {start, end});

    while (i < n) out.add(intervals[i++]);
    return out.toArray(new int[0][]);
}
```

Cost. $O(n)$ time — the input is already sorted, so do not re-sort it.

Trap. Adding the merged interval inside the second loop, which emits it once per overlap. It goes in exactly once, after the loop.

P30. Sort employees by salary then name

Easy

Sort records by salary descending, and alphabetically for equal salaries. A pure comparator drill — this exact shape appears in data-processing tasks.

Idea. The `Comparator` factory methods compose. Build the ordering declaratively rather than hand-rolling the branches.

Solution

```java
record Employee(String name, int salary) {}

public static void sortEmployees(List<Employee> staff) {
    staff.sort(Comparator.comparingInt(Employee::salary).reversed()
                         .thenComparing(Employee::name));
}

// the hand-written equivalent, when you cannot remember the fluent form
public static void sortEmployeesManually(List<Employee> staff) {
    staff.sort((a, b) -> {
        if (a.salary() != b.salary())
            return Integer.compare(b.salary(), a.salary());
        return a.name().compareTo(b.name());
    });
}
```

Cost. $O(n \log n)$.

Trap. `.reversed()` reverses *everything composed so far*. Chaining it on after `thenComparing(name)` reverses the name order as well. Put `reversed()` on the salary key alone, as above.

P31. Kth largest element

Medium

Return the kth largest value in an unsorted array (the kth in sorted descending order, duplicates counted).

Idea. Keep a min-heap of size k. The smallest thing in it is, once you have seen everything, the kth largest overall.

Solution

```java
public static int kthLargest(int[] a, int k) {
    PriorityQueue<Integer> heap = new PriorityQueue<>();   // min-heap
    for (int x : a) {
        heap.offer(x);
        if (heap.size() > k) heap.poll();
    }
    return heap.peek();
}
```

Cost. $O(n \log k)$ time, $O(k)$ space. Sorting is $O(n \log n)$ and perfectly acceptable when k is close to n.

Trap. Using a max-heap and polling k times is $O(n + k \log n)$ to build and drain — fine, but it needs all n elements in memory. The size-k min-heap is the answer when the input is a stream.

6. Matrices

Six problems. Two-dimensional arrays are mostly an index-discipline test: get `rows` and `cols` the right way round and most of the difficulty disappears.

P32. Rotate a square matrix 90 degrees clockwise

Medium

Rotate in place. `[[1,2],[3,4]]` becomes `[[3,1],[4,2]]`.

Idea. Transpose (reflect across the main diagonal), then reverse each row. Two simple passes beat one confusing index formula.

Solution

```
public static void rotate90(int[][] m) {
    int n = m.length;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++) {      // j starts at i+1
            int t = m[i][j]; m[i][j] = m[j][i]; m[j][i] = t;
        }
    for (int[] row : m)
        for (int i = 0, j = n - 1; i < j; i++, j--) {
            int t = row[i]; row[i] = row[j]; row[j] = t;
        }
}
```

Cost. $O(n^2)$ time, $O(1)$ space.

Trap. Starting the inner transpose loop at `j = 0` swaps every pair twice and leaves the matrix unchanged. It must start at `i + 1`.

P33. Spiral traversal

Medium

Return the elements in spiral order: left to right along the top, down the right side, right to left along the bottom, up the left side, then inwards.

Idea. Four moving boundaries. After walking each edge, shrink that boundary, and check before the two "return" edges that the boundaries have not crossed.

Solution

```
public static List<Integer> spiral(int[][] m) {
    List<Integer> out = new ArrayList<>();
    if (m == null || m.length == 0) return out;
    int top = 0, bottom = m.length - 1;
    int left = 0, right = m[0].length - 1;

    while (top <= bottom && left <= right) {
        for (int c = left; c <= right; c++) out.add(m[top][c]);
        top++;
        for (int r = top; r <= bottom; r++) out.add(m[r][right]);
        right--;
        if (top <= bottom) {
            for (int c = right; c >= left; c--) out.add(m[bottom][c]);
            bottom--;
        }
        if (left <= right) {
            for (int r = bottom; r >= top; r--) out.add(m[r][left]);
            left++;
        }
    }
    return out;
}
```

Cost. $O(\text{rows} \times \text{cols})$ time.

Trap. Omitting the two guards duplicates a row or a column on non-square matrices — a single remaining row would be walked both left-to-right and right-to-left.

P34. Count islands

Medium

The grid holds `'1'` for land and `'0'` for water. Count the connected groups of land, using four-way adjacency.

Idea. Scan every cell. The first time you touch a piece of land, increment the count and flood the whole island so it is never counted again.

Solution

```java
public static int countIslands(char[][] grid) {
    if (grid == null || grid.length == 0) return 0;
    int rows = grid.length, cols = grid[0].length, islands = 0;
    int[][] dirs = {{-1,0},{1,0},{0,-1},{0,1}};

    for (int r = 0; r < rows; r++)
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] != '1') continue;
            islands++;
            Deque<int[]> queue = new ArrayDeque<>();
            queue.add(new int[] {r, c});
            grid[r][c] = '0';                    // mark on push
            while (!queue.isEmpty()) {
                int[] cell = queue.poll();
                for (int[] d : dirs) {
                    int nr = cell[0] + d[0], nc = cell[1] + d[1];
                    if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue;
                    if (grid[nr][nc] != '1') continue;
                    grid[nr][nc] = '0';
                    queue.add(new int[] {nr, nc});
                }
            }
        }
    return islands;
}
```

Cost. $O(\text{rows} \times \text{cols})$ time and space.

Trap. Marking a cell visited when you *pop* it rather than when you *push* it lets the same cell enter the queue many times, and a large open grid then blows up. Mark on push, always.

P35. Set matrix zeroes

Medium

If any cell is zero, set its whole row and column to zero. The catch is doing it without a second matrix.

Idea. Two passes. First record *which* rows and columns must be cleared; only then clear them. Clearing as you go turns the whole matrix to zero.

Solution

```java
public static void setZeroes(int[][] m) {
    if (m == null || m.length == 0) return;
    int rows = m.length, cols = m[0].length;
    boolean[] clearRow = new boolean[rows];
    boolean[] clearCol = new boolean[cols];

    for (int r = 0; r < rows; r++)
        for (int c = 0; c < cols; c++)
            if (m[r][c] == 0) { clearRow[r] = true; clearCol[c] = true; }
```

```
10
11      for (int r = 0; r < rows; r++)
12          for (int c = 0; c < cols; c++)
13              if (clearRow[r] || clearCol[c]) m[r][c] = 0;
14  }
```

Cost. $O(\text{rows} \times \text{cols})$ time, $O(\text{rows} + \text{cols})$ space.

Trap. The naive one-pass version writes zeros that the rest of the pass then reads as original zeros, cascading until everything is zero. The $O(1)$-space variant stores the flags in row 0 and column 0 and needs a separate flag for each of those — more clever than it is worth under time pressure.

P36. Diagonal sums

Easy

Sum the main diagonal and the anti-diagonal of a square matrix, counting the centre of an odd-sized matrix once.

Idea. One loop. At step i the two diagonal cells are $m[i][i]$ and $m[i][n-1-i]$; they coincide exactly when $i = n-1-i$.

Solution

```
1  public static long diagonalSum(int[][] m) {
2      int n = m.length;
3      long total = 0;
4      for (int i = 0; i < n; i++) {
5          total += m[i][i];
6          if (i != n - 1 - i) total += m[i][n - 1 - i];
7      }
8      return total;
9  }
```

Cost. $O(n)$ time, $O(1)$ space.

Trap. Double-counting the centre cell of an odd matrix. And the total can exceed `int` — use `long`.

P37. Search a row-and-column sorted matrix

Medium

Every row is sorted left to right and every column top to bottom. Decide whether a target is present.

Idea. Start at the top-right corner. That cell is the largest in its row and the smallest in its column, so a comparison always eliminates an entire row or an entire column.

Solution

```
1  public static boolean contains(int[][] m, int target) {
2      if (m == null || m.length == 0) return false;
3      int r = 0, c = m[0].length - 1;
4      while (r < m.length && c >= 0) {
5          if (m[r][c] == target) return true;
6          if (m[r][c] > target) c--;      // whole column is too big
7          else r++;                       // whole row is too small
8      }
9      return false;
10 }
```

Cost. $O(\text{rows} + \text{cols})$ time, $O(1)$ space.

Trap. Starting at the top-left corner gives no information: both moves lead to larger values, so you cannot eliminate anything. The corner choice is the entire trick.

7. Recursion, BFS and DFS

Seven problems. Recursion is fine for trees, where depth is logarithmic. On grids, use an explicit queue — depth there is measured in tens of thousands.

P38. Fibonacci with memoisation

Easy

Return the nth Fibonacci number. The point is the memo, not the formula.

Idea. The naive recursion recomputes the same subproblems exponentially often. Cache each answer the first time you compute it. Better still, note that only two previous values are ever needed and drop the recursion entirely.

Solution

```java
public static long fib(int n) {
    if (n < 2) return n;
    long prev = 0, current = 1;
    for (int i = 2; i <= n; i++) {
        long next = prev + current;
        prev = current;
        current = next;
    }
    return current;
}

// the memoised recursion, when the recurrence is not so easily unrolled
public static long fibMemo(int n, Map<Integer,Long> memo) {
    if (n < 2) return n;
    Long cached = memo.get(n);
    if (cached != null) return cached;
    long value = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
    memo.put(n, value);
    return value;
}
```

Cost. Iterative: $O(n)$ time, $O(1)$ space. Memoised: $O(n)$ both.

Trap. `int` overflows at `fib(47)`. If the statement does not bound n, use `long` and say so.

P39. Maximum depth of a binary tree

Easy

Return the number of nodes on the longest root-to-leaf path.

Idea. The depth of a node is one more than the deeper of its two subtrees. This is the template every tree recursion follows: base case for `null`, combine the children.

Solution

```java
static class Node {
    int value;
    Node left, right;
}

public static int maxDepth(Node root) {
    if (root == null) return 0;
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}
```

Cost. $O(n)$ time, $O(h)$ stack for height h.

Trap. Returning `1` rather than `0` for a `null` node counts a missing child as a level and gives every leaf a depth of 2.

P40. Level-order traversal Medium

Return the node values grouped by depth, top level first.

Idea. BFS with a queue, taking a whole level at a time: record the queue size before the loop, then pop exactly that many nodes. That snapshot is what separates one level from the next.

Solution

```java
public static List<List<Integer>> levelOrder(Node root) {
    List<List<Integer>> out = new ArrayList<>();
    if (root == null) return out;
    Deque<Node> queue = new ArrayDeque<>();
    queue.add(root);
    while (!queue.isEmpty()) {
        int levelSize = queue.size();            // snapshot
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < levelSize; i++) {
            Node node = queue.poll();
            level.add(node.value);
            if (node.left != null) queue.add(node.left);
            if (node.right != null) queue.add(node.right);
        }
        out.add(level);
    }
    return out;
}
```

Cost. $O(n)$ time, $O(w)$ space for the widest level w.

Trap. Writing `for (int i = 0; i < queue.size(); i++)` re-evaluates the size as children are added, so levels bleed into each other. Take the snapshot into a variable.

P41. Flood fill Easy

Starting at a pixel, repaint it and every same-coloured pixel connected to it (four-way) with a new colour.

Idea. BFS from the start pixel. Guard against the new colour equalling the old one, or the fill never terminates.

Solution

```java
public static int[][] floodFill(int[][] image, int sr, int sc, int colour) {
    int old = image[sr][sc];
    if (old == colour) return image;             // the essential guard
    int rows = image.length, cols = image[0].length;
    int[][] dirs = {{-1,0},{1,0},{0,-1},{0,1}};

    Deque<int[]> queue = new ArrayDeque<>();
    queue.add(new int[] {sr, sc});
    image[sr][sc] = colour;
    while (!queue.isEmpty()) {
        int[] cell = queue.poll();
        for (int[] d : dirs) {
            int r = cell[0] + d[0], c = cell[1] + d[1];
            if (r < 0 || r >= rows || c < 0 || c >= cols) continue;
            if (image[r][c] != old) continue;
            image[r][c] = colour;
            queue.add(new int[] {r, c});
        }
    }
    return image;
}
```

Cost. $O(\text{rows} \times \text{cols})$.

Trap. Without the `old == colour` early return, every repainted cell still matches the target colour, so it is enqueued again forever.

Also appears as: The engine behind Task 3 of the assessment.

P42. All permutations of a string

Medium

Return every ordering of the characters. For n characters there are $n!$ of them, so this is only ever asked for small inputs.

Idea. Backtracking. Choose an unused character, recurse, then put it back. The "put it back" line is what makes it explore every branch rather than one.

Solution

```
public static List<String> permutations(String s) {
    List<String> out = new ArrayList<>();
    if (s == null || s.isEmpty()) return out;
    build(s.toCharArray(), new boolean[s.length()],
          new StringBuilder(), out);
    return out;
}

private static void build(char[] chars, boolean[] used,
                          StringBuilder current, List<String> out) {
    if (current.length() == chars.length) {
        out.add(current.toString());
        return;
    }
    for (int i = 0; i < chars.length; i++) {
        if (used[i]) continue;
        used[i] = true;
        current.append(chars[i]);
        build(chars, used, current, out);
        current.deleteCharAt(current.length() - 1);   // undo
        used[i] = false;                              // undo
    }
}
```

Cost. $O(n \cdot n!)$ time.

Trap. Forgetting either undo. The `StringBuilder` and the `used` array are shared across the whole search, so every change must be reversed on the way out.

P43. All subsets

Medium

Return every subset of a set of distinct numbers — 2^n of them.

Idea. Each element is either in or out, so the subsets map one-to-one onto the binary numbers from 0 to $2^n - 1$. Bit i set means element i is included. No recursion needed.

Solution

```
public static List<List<Integer>> subsets(int[] a) {
    List<List<Integer>> out = new ArrayList<>();
    int total = 1 << a.length;                    // 2^n
    for (int mask = 0; mask < total; mask++) {
        List<Integer> subset = new ArrayList<>();
        for (int i = 0; i < a.length; i++)
            if ((mask & (1 << i)) != 0) subset.add(a[i]);
        out.add(subset);
    }
    return out;
}
```

Cost. $O(n \cdot 2^n)$ time.

Trap. `1 << a.length` overflows once the array has 31 elements. That is academic here — 2^{31} subsets would not fit in memory anyway — but the shift is the thing that breaks first.

P44. Shortest path in a grid with obstacles

Medium

Cells are open (`0`) or blocked (`1`). Return the fewest steps from the top-left to the bottom-right, moving four-way, or -1.

Idea. BFS, not DFS. On an unweighted graph BFS reaches every cell by a shortest path, so the first time you arrive at the target you are done.

Solution

```java
public static int shortestPath(int[][] grid) {
    if (grid == null || grid.length == 0) return -1;
    int rows = grid.length, cols = grid[0].length;
    if (grid[0][0] == 1 || grid[rows - 1][cols - 1] == 1) return -1;
    int[][] dirs = {{-1,0},{1,0},{0,-1},{0,1}};

    boolean[][] seen = new boolean[rows][cols];
    Deque<int[]> queue = new ArrayDeque<>();
    queue.add(new int[] {0, 0});
    seen[0][0] = true;
    int steps = 1;

    while (!queue.isEmpty()) {
        int levelSize = queue.size();
        for (int i = 0; i < levelSize; i++) {
            int[] cell = queue.poll();
            if (cell[0] == rows - 1 && cell[1] == cols - 1) return steps;
            for (int[] d : dirs) {
                int r = cell[0] + d[0], c = cell[1] + d[1];
                if (r < 0 || r >= rows || c < 0 || c >= cols) continue;
                if (seen[r][c] || grid[r][c] == 1) continue;
                seen[r][c] = true;
                queue.add(new int[] {r, c});
            }
        }
        steps++;
    }
    return -1;
}
```

Cost. $O(\text{rows} \times \text{cols})$ time and space.

Trap. DFS finds *a* path, not the shortest one, and people reach for it because recursion is quicker to type. If the question says shortest and the edges are unweighted, it is BFS.

8. Stacks and queues

Five problems. A stack is the right answer whenever the most recent unresolved thing is the one you need next.

P45. Valid parentheses

Easy

Given a string of `()`, `[]` and `{}`, decide whether every bracket is closed by the matching kind, in the right order.

Idea. Push openers. On a closer, the top of the stack must be its partner. At the end the stack must be empty.

Solution

```
public static boolean isValid(String s) {
    if (s == null) return true;
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        switch (c) {
            case '(' : case '[' : case '{' : stack.push(c); break;
            case ')' : if (stack.isEmpty() || stack.pop() != '(') return false; break;
            case ']' : if (stack.isEmpty() || stack.pop() != '[') return false; break;
            case '}' : if (stack.isEmpty() || stack.pop() != '{') return false; break;
            default  : break;    // ignore anything else
        }
    }
    return stack.isEmpty();
}
```

Cost. $O(n)$ time, $O(n)$ space.

Trap. Two endings get forgotten. Popping an empty stack on a stray closer throws, so test `isEmpty()` first; and leaving the stack non-empty at the end means unclosed openers, which is also invalid.

P46. Evaluate reverse Polish notation

Medium

Evaluate tokens like `["2","1","+","3","*"]`, which means $(2 + 1) \times 3 = 9$.

Idea. Push numbers. On an operator, pop two, apply, push the result. The first value popped is the right-hand operand.

Solution

```
public static int evalRPN(String[] tokens) {
    Deque<Integer> stack = new ArrayDeque<>();
    for (String t : tokens) {
        switch (t) {
            case "+": case "-": case "*": case "/": {
                int right = stack.pop();           // order matters
                int left  = stack.pop();
                stack.push(apply(left, right, t));
                break;
            }
            default:
                stack.push(Integer.parseInt(t));
        }
    }
    return stack.pop();
}

private static int apply(int a, int b, String op) {
    switch (op) {
```

```
        case "+": return a + b;
        case "-": return a - b;
        case "*": return a * b;
        default : return a / b;
    }
}
```

Cost. $O(n)$ time, $O(n)$ space.

Trap. Popping the operands in the wrong order. It is invisible for `+` and `*` and wrong for `-` and `/`, so the bug survives your first two test cases.

P47. Next greater element

Medium

For every element, find the first element to its right that is larger, or -1. `[2,1,2,4]` gives `[4,2,4,-1]`.

Idea. A monotonic stack of *indices* still waiting for an answer. When a new value arrives, it resolves every waiting index smaller than it. Each index is pushed and popped once.

Solution

```
public static int[] nextGreater(int[] a) {
    int n = a.length;
    int[] out = new int[n];
    Arrays.fill(out, -1);
    Deque<Integer> waiting = new ArrayDeque<>();   // indices

    for (int i = 0; i < n; i++) {
        while (!waiting.isEmpty() && a[waiting.peek()] < a[i])
            out[waiting.pop()] = a[i];
        waiting.push(i);
    }
    return out;
}
```

Cost. $O(n)$ time — the inner `while` is amortised, since every index leaves the stack at most once. $O(n)$ space.

Trap. The nested-loop version is $O(n^2)$ and looks so natural that people write it without noticing. Also store indices, not values, or you cannot write the answer back to the right slot.

Also appears as: "Daily temperatures", where the answer is the gap `i - j` rather than the value.

P48. Min stack

Medium

A stack that also reports its minimum in $O(1)$.

Idea. Keep a second stack holding the minimum *as of* each push. Pushing records the smaller of the new value and the current minimum; popping discards both in step, so the two stacks stay aligned.

Solution

```
class MinStack {
    private final Deque<Integer> values = new ArrayDeque<>();
    private final Deque<Integer> minima = new ArrayDeque<>();

    void push(int x) {
        values.push(x);
        minima.push(minima.isEmpty() ? x : Math.min(x, minima.peek()));
    }

    int pop() {
        minima.pop();
        return values.pop();
```

```
    }

    int top()     { return values.peek(); }
    int getMin()  { return minima.peek(); }
}
```

Cost. $O(1)$ for every operation, $O(n)$ space.

Trap. Keeping a single `int min` field instead. It is right until you pop the minimum, and then there is no way to recover the previous one without scanning.

P49. Simplify a Unix path

Medium

Turn `"/a/./b/../../c/"` into `"/c"`. Handle `.`, `..` and repeated slashes.

Idea. Split on slashes and push each real component. `..` pops, `.` and empty components are skipped. Join what is left.

Solution

```
public static String simplify(String path) {
    Deque<String> parts = new ArrayDeque<>();      // used as a list
    for (String part : path.split("/")) {
        if (part.isEmpty() || part.equals(".")) continue;
        if (part.equals("..")) parts.pollLast();  // no-op when empty
        else parts.addLast(part);
    }
    if (parts.isEmpty()) return "/";
    StringBuilder out = new StringBuilder();
    for (String part : parts) out.append('/').append(part);
    return out.toString();
}
```

Cost. $O(n)$ time, $O(n)$ space.

Trap. `..` at the root must do nothing, not throw — `pollLast` returns `null` on an empty deque where `removeLast` throws. And the result for an empty stack is `"/"`, not `""`.

9. Data processing

Six problems in the shape candidates actually report from consulting-tech screens: parse records, validate them, group them, aggregate them, report the top few. Less algorithmic than the rest of the book and more likely to be what you face.

P50. Average salary by department

Medium

Records are `"name,department,salary"`. Return the top n departments by average salary as `"department(average)"`, average descending then name ascending. The average is integer division. Ignore any record that does not have three fields, has an empty department, or whose salary is not a whole number at least zero.

Idea. Accumulate a *total and a count* per department, never a running average. Compute each average once, at the end, from exact numbers.

Solution

```
public static String[] topDepartments(String[] records, int n) {
    if (records == null || n <= 0) return new String[0];
    Map<String,long[]> tally = new HashMap<>();      // [total, count]

    for (String record : records) {
        if (record == null) continue;
        String[] parts = record.split(",", -1);      // keep empty fields
        if (parts.length != 3) continue;
        String dept = parts[1].trim();
        if (dept.isEmpty()) continue;
        long salary;
        try { salary = Long.parseLong(parts[2].trim()); }
        catch (NumberFormatException e) { continue; }
        if (salary < 0) continue;

        long[] slot = tally.computeIfAbsent(dept, k -> new long[2]);
        slot[0] += salary;
        slot[1]++;
    }
    if (tally.isEmpty()) return new String[0];

    List<String> names = new ArrayList<>(tally.keySet());
    names.sort((a, b) -> {
        long avgA = tally.get(a)[0] / tally.get(a)[1];
        long avgB = tally.get(b)[0] / tally.get(b)[1];
        if (avgA != avgB) return Long.compare(avgB, avgA);
        return a.compareTo(b);
    });

    int size = Math.min(n, names.size());
    String[] out = new String[size];
    for (int i = 0; i < size; i++) {
        long[] slot = tally.get(names.get(i));
        out[i] = names.get(i) + "(" + (slot[0] / slot[1]) + ")";
    }
    return out;
}
```

Cost. $O(R + D \log D)$ for R records and D departments.

Trap. Four of them, and each one is a real submission killer. `split(",")` without `-1` silently drops trailing empty fields, so `"a,x,"` looks like a two-field record instead of an invalid three-field one. An `int` total overflows. Averaging averages is wrong whenever departments differ in size. And a

negative salary parses perfectly well, so `parseInt` succeeding is not the same as the record being valid.

Also appears as: Task 2 of the assessment.

P51. Filter and total transactions

Medium

Records are `"id,category,amount"` with the amount in cents. Return the total per category, but only for categories whose total is at least a threshold, sorted by total descending.

Idea. One grouping pass, then filter, then sort. Doing the filter before the grouping is finished is the common error — a category only crosses the threshold once all its rows are in.

Solution

```
public static List<String> categoriesOver(String[] rows, long threshold) {
    Map<String,Long> totals = new HashMap<>();
    for (String row : rows) {
        if (row == null) continue;
        String[] parts = row.split(",", -1);
        if (parts.length != 3) continue;
        try {
            totals.merge(parts[1].trim(),
                         Long.parseLong(parts[2].trim()), Long::sum);
        } catch (NumberFormatException ignored) { }
    }

    List<Map.Entry<String,Long>> kept = new ArrayList<>();
    for (Map.Entry<String,Long> e : totals.entrySet())
        if (e.getValue() >= threshold) kept.add(e);

    kept.sort((a, b) -> b.getValue().compareTo(a.getValue()));

    List<String> out = new ArrayList<>();
    for (Map.Entry<String,Long> e : kept)
        out.add(e.getKey() + "=" + e.getValue());
    return out;
}
```

Cost. $O(R + C \log C)$.

Trap. Filtering inside the parsing loop. Also, `b.getValue().compareTo` on boxed `Long` is fine, but `(int)(b.getValue() - a.getValue())` is not — the subtraction can overflow and the cast then reverses the order.

P52. Validate records and report the errors

Medium

Given records `"id,email,age"`, return one message per invalid record in input order: `"line 3: bad age"`. A record is invalid if it has the wrong field count, an email with no `@`, or an age outside 0 to 150. Report only the first problem found in each record.

Idea. A validation ladder: check in a fixed order and return the first failure. Keep the line number from the loop index, counting from 1.

Solution

```
public static List<String> validate(String[] rows) {
    List<String> errors = new ArrayList<>();
    for (int i = 0; i < rows.length; i++) {
        String problem = firstProblem(rows[i]);
        if (problem != null) errors.add("line " + (i + 1) + ": " + problem);
    }
    return errors;
}

```

```
private static String firstProblem(String row) {
    if (row == null) return "missing record";
    String[] parts = row.split(",", -1);
    if (parts.length != 3) return "wrong field count";
    if (!parts[1].contains("@")) return "bad email";
    int age;
    try { age = Integer.parseInt(parts[2].trim()); }
    catch (NumberFormatException e) { return "bad age"; }
    if (age < 0 || age > 150) return "bad age";
    return null;
}
```

Cost. $O(R)$.

Trap. Line numbers are one-based in every message format anyone asks for, and the loop index is zero-based. Off by one here fails every test at once, which at least makes it easy to spot.

P53. Join two datasets on a key

Medium

One array holds `"userId,name"`, another holds `"userId,orderTotal"`. Return `"name:total"` for every user present in both, summing a user's orders, sorted by total descending.

Idea. Index the smaller side into a map, then stream the larger side against it. This is a hash join, and it is $O(n+m)$ where the nested-loop version is $O(nm)$.

Solution

```
public static List<String> join(String[] users, String[] orders) {
    Map<String,String> nameById = new HashMap<>();
    for (String u : users) {
        String[] p = u.split(",", -1);
        if (p.length == 2) nameById.put(p[0].trim(), p[1].trim());
    }

    Map<String,Long> totalById = new HashMap<>();
    for (String o : orders) {
        String[] p = o.split(",", -1);
        if (p.length != 2) continue;
        String id = p[0].trim();
        if (!nameById.containsKey(id)) continue;      // inner join
        try { totalById.merge(id, Long.parseLong(p[1].trim()), Long::sum); }
        catch (NumberFormatException ignored) { }
    }

    List<Map.Entry<String,Long>> rows = new ArrayList<>(totalById.entrySet());
    rows.sort((a, b) -> b.getValue().compareTo(a.getValue()));

    List<String> out = new ArrayList<>();
    for (Map.Entry<String,Long> e : rows)
        out.add(nameById.get(e.getKey()) + ":" + e.getValue());
    return out;
}
```

Cost. $O(n+m)$ plus the sort.

Trap. Deciding what kind of join you were asked for. This one drops users with no orders and orders with no user. A left join would keep every user with a total of zero — a different answer, and the statement will say which.

P54. Bucket events by hour and find the peak

Medium

Each record is `"eventId,timestampSeconds"`. Count the events falling in each hour and return the busiest hour as an hour index, breaking ties by the earlier hour.

Idea. The bucket key is `timestamp / 3600` — integer division is the bucketing. Then a single scan for the maximum, taking `>` and not `>=` so the earliest hour wins a tie.

Solution

```
public static long busiestHour(String[] events) {
    Map<Long,Integer> perHour = new HashMap<>();
    for (String e : events) {
        String[] p = e.split(",", -1);
        if (p.length != 2) continue;
        try {
            long hour = Long.parseLong(p[1].trim()) / 3600;
            perHour.merge(hour, 1, Integer::sum);
        } catch (NumberFormatException ignored) { }
    }
    if (perHour.isEmpty()) return -1;

    long bestHour = Long.MAX_VALUE;
    int bestCount = -1;
    for (Map.Entry<Long,Integer> e : perHour.entrySet()) {
        if (e.getValue() > bestCount
                || (e.getValue() == bestCount && e.getKey() < bestHour)) {
            bestCount = e.getValue();
            bestHour = e.getKey();
        }
    }
    return bestHour;
}
```

Cost. $O(R)$.

Trap. Relying on iteration order to settle ties. `HashMap` order is unspecified and changes with the keys, so the tie-break has to be written out explicitly, as in the second clause above.

P55. Running balance per account

Medium

Transactions arrive as `"account,amount"` in chronological order. Return, for each account, the lowest balance it ever reached.

Idea. One pass with two maps, or one map of a small array. Update the running balance, then compare it to the stored minimum — in that order, because the minimum has to include the transaction you just applied.

Solution

```
public static Map<String,Long> lowestBalances(String[] transactions) {
    Map<String,Long> balance = new HashMap<>();
    Map<String,Long> lowest  = new HashMap<>();

    for (String t : transactions) {
        String[] p = t.split(",", -1);
        if (p.length != 2) continue;
        String account = p[0].trim();
        long amount;
        try { amount = Long.parseLong(p[1].trim()); }
        catch (NumberFormatException e) { continue; }

        long now = balance.getOrDefault(account, 0L) + amount;
        balance.put(account, now);
        lowest.merge(account, now, Math::min);
    }
    return lowest;
}
```

Cost. $O(R)$ time, $O(A)$ space for A accounts.

Trap. Seeding the minimum at `0` for every account. If an account only ever receives money its lowest balance is its first deposit, not zero — and `merge` with no prior value stores the first balance, which is exactly right.

10. Stretch

Five problems above the bar. Nothing here is likely in a seventy-minute screen for a final-year internship, and two of them were cut from the assessment for exactly that reason. Work them when the first eight chapters are comfortable — the point is that a hard problem on the day should feel familiar rather than alarming.

P56. Minimum travel time with k free roads

Hard

Cities $0 \dots n-1$ are joined by bidirectional roads $\{u, v, t\}$. You hold k passes; using one on a road makes that road cost 0. Return the least time from city 0 to city $n-1$, or -1.

Idea. The state you search is not a city, it is a *pair*: the city, and how many passes you have spent. Build $k+1$ copies of the graph — layers — and run one Dijkstra over the lot. From (u, p) each road either costs its time and keeps you in layer p, or costs nothing and drops you into layer $p+1$.

Solution

```
public static int minTravelTime(int n, int[][] roads, int k) {
    if (n == 1) return 0;
    int layers = (int) Math.max(0L, Math.min((long) k, (long) n - 1));

    List<int[]>[] adj = new List[n];
    for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
    for (int[] r : roads) {
        adj[r[0]].add(new int[] {r[1], r[2]});
        if (r[0] != r[1]) adj[r[1]].add(new int[] {r[0], r[2]});
    }

    long[][] dist = new long[n][layers + 1];
    for (long[] row : dist) Arrays.fill(row, Long.MAX_VALUE);
    dist[0][0] = 0;

    // {cost, city, passesUsed}
    PriorityQueue<long[]> pq =
        new PriorityQueue<>((a, b) -> Long.compare(a[0], b[0]));
    pq.add(new long[] {0, 0, 0});

    while (!pq.isEmpty()) {
        long[] top = pq.poll();
        long cost = top[0];
        int u = (int) top[1], used = (int) top[2];
        if (cost > dist[u][used]) continue;          // stale entry
        if (u == n - 1) return (int) cost;

        for (int[] edge : adj[u]) {
            int v = edge[0];
            long paid = cost + edge[1];
            if (paid < dist[v][used]) {
                dist[v][used] = paid;
                pq.add(new long[] {paid, v, used});
            }
            if (used < layers && cost < dist[v][used + 1]) {
                dist[v][used + 1] = cost;
                pq.add(new long[] {cost, v, used + 1});
            }
        }
    }
    return -1;
}
```

Cost. $O(E \cdot K \log(V \cdot K))$ with $K = \min(k, n-1) + 1$.

Trap. Three. k can be 10^9, so cap the layers at $n-1$ — a simple path uses no more roads than that, and without the cap you allocate a billion rows. Distances overflow `int`, so accumulate in `long`. And the tempting greedy — find the shortest path, then zero its k heaviest edges — is simply wrong: the best route when you have passes is often a different route entirely.

Also appears as: "Cheapest flights with k stops" and "minimum cost path with k refuels" are the same layered-state idea.

P57. Expiring files

Hard

Extend the Part B `CloudStorage` so files can be given a lifetime. `addFileAt(timestamp, user, name, size, ttl)` creates a file alive over $[timestamp, timestamp + ttl)$; `getFileSizeAt`, `deleteFileAt` and `getNLargestAt` see only files alive at their timestamp. An expired file frees its owner's capacity.

Idea. The level attacks the incremental `used` counter that made the earlier levels cheap: bytes now free themselves with no operation to hook. Rather than recompute usage on every query, make expiry *lazy* — each timestamped call first deletes everything already dead at its timestamp, through the ordinary delete path, and then delegates to the untimed method that already exists. One notion of "the files that exist", and every earlier level untouched.

Solution

```
// FileEntry gains: final long expiresAt;   (Long.MAX_VALUE = never)

public Integer addFileAt(int timestamp, String user,
                         String name, int size, int ttl) {
    if (ttl <= 0 || timestamp < 0) return null;
    purge(timestamp);
    // long, because timestamp + ttl overflows int near Integer.MAX_VALUE
    return addEntry(user, name, size, (long) timestamp + (long) ttl);
}

public Integer getFileSizeAt(int timestamp, String name) {
    purge(timestamp);
    return getFileSize(name);
}

/** Drops every file dead at `now`, crediting each owner's capacity. */
private void purge(long now) {
    List<String> expired = null;
    for (Map.Entry<String,FileEntry> e : files.entrySet())
        if (e.getValue().expiresAt <= now) {
            if (expired == null) expired = new ArrayList<>();
            expired.add(e.getKey());
        }
    if (expired == null) return;
    for (String name : expired) removeEntry(name);   // credits the owner
}
```

Cost. $O(F)$ per timestamped call in the worst case, $O(1)$ amortised per file since each file is purged once.

Trap. Collecting the dead keys before deleting — removing from `files` while iterating it throws `ConcurrentModificationException`. Storing an *absolute* expiry rather than a TTL, so backup and restore do not restart a lifetime. And treating the interval as closed: a file with `ttl` 10 created at 0 is dead *at* 10, not after it.

P58. LRU cache

Hard

A fixed-capacity cache with $O(1)$ `get` and `put`. When it is full, the least recently used entry is evicted. A `get` counts as a use.

Idea. `LinkedHashMap` does this in a dozen lines: construct it in access-order mode and override `removeEldestEntry`. Know the manual version too — a hash map to nodes plus a doubly linked list — because interviewers sometimes ban the shortcut.

Solution

```java
class LruCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;

    LruCache(int capacity) {
        // initialCapacity, loadFactor, accessOrder=true
        super(16, 0.75f, true);
        this.capacity = capacity;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;
    }
}
```

Cost. $O(1)$ expected for both operations.

Trap. The third constructor argument. Left at `false` — the default — you get insertion order, which evicts the oldest *inserted* entry and makes it an FIFO cache, not an LRU one. The bug passes any test that never re-reads a key.

P59. Coin change

Hard

Given coin denominations and a target, return the fewest coins that sum to it, or -1. Each denomination may be used any number of times.

Idea. Bottom-up dynamic programming over the amount. The best way to make a is one coin on top of the best way to make $a - \mathit{coin}$, minimised over the coins. Fill the table upwards so every subproblem is already solved when you need it.

Solution

```java
public static int coinChange(int[] coins, int amount) {
    if (amount < 0) return -1;
    int[] best = new int[amount + 1];
    Arrays.fill(best, Integer.MAX_VALUE);
    best[0] = 0;                                // zero coins make zero

    for (int a = 1; a <= amount; a++)
        for (int coin : coins) {
            if (coin <= 0 || coin > a) continue;
            if (best[a - coin] == Integer.MAX_VALUE) continue;
            best[a] = Math.min(best[a], best[a - coin] + 1);
        }

    return best[amount] == Integer.MAX_VALUE ? -1 : best[amount];
}
```

Cost. $O(\mathit{amount} \times \mathit{coins})$ time, $O(\mathit{amount})$ space.

Trap. The greedy "take the biggest coin that fits" is wrong on coin sets like $\{1, 3, 4\}$, where making 6 greedily takes $4 + 1 + 1$ and the answer is $3 + 3$. It happens to work on real currency, which is why people trust it. Also guard `best[a - coin] == MAX_VALUE` before adding one, or it overflows to a negative and poisons the table.

P60. Longest increasing subsequence

Hard

Return the length of the longest strictly increasing subsequence — the elements need not be adjacent. `[10,9,2,5,3,7,101,18]` gives 4.

Idea. Keep a list `tails`, where `tails[i]` is the smallest value that can end an increasing subsequence of length $i+1$. That list is always sorted, so each new element is placed by binary search: it either extends the list or lowers an existing tail, making future extensions easier. The list is not itself a valid subsequence — only its *length* is the answer.

Solution

```
public static int lengthOfLIS(int[] a) {
    if (a == null || a.length == 0) return 0;
    int[] tails = new int[a.length];
    int size = 0;

    for (int x : a) {
        int lo = 0, hi = size;
        while (lo < hi) {                       // first tail >= x
            int mid = (lo + hi) >>> 1;
            if (tails[mid] < x) lo = mid + 1;
            else hi = mid;
        }
        tails[lo] = x;
        if (lo == size) size++;
    }
    return size;
}
```

Cost. $O(n \log n)$ time, $O(n)$ space. The straightforward DP is $O(n^2)$ and is the right answer below a few thousand elements.

Trap. `(lo + hi) / 2` overflows for huge bounds; `>> 1` is the habit worth having. And for a *non-strictly* increasing subsequence the binary search changes to "first tail > x" — a one-character difference that silently changes the answer.